Git Challenges

1. Resolve Merge Conflicts

Create a merge conflict intentionally (two users editing the same line).

Resolve the conflict and push the changes.

Step 1:- write some text in any file then add It to tracking

Step 2:- Commit this changes

Step 3:- Then switch to different file and work on same file. Add file to tracking and commit the changes

Step 4:- Now go to the base branch where we want to merge and use git merge branch_name.

Step 5:- We get error of merge conflict. And it enters into merging state.

Step 6:- Edit the file remove text from any branch and save the changes.

Step 7:- After we commit the changes it will automatically merge.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ ls
Git.txt  Text  greet  hello

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ echo "Main branch" > Git.txt

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git add Git.txt
warning: in the working copy of 'Git.txt', LF will be repla

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git commit -m "Changes in git in main branch"
[main 36280d8] Changes in git in main branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git switch release
Switched to branch 'release'

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (release)
$ echo "Release branch" > Git.txt

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (release)
$ git add Git.txt
warning: in the working copy of 'Git.txt', LF will be replac

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (release)
$ git commit -m "Changes in git in Release branch"
[release cad1348] Changes in git in Release branch
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (release)
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git merge release
Auto-merging Git.txt
CONFLICT (content): Merge conflict in Git.txt
Automatic merge failed; fix conflicts and then commit the result.
```

UCRT64:/d/Git/Git-practice

```
<<<<<<< HEAD
Main branch
=======
Release branch
>>>>>>> release
~
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|MERGING)
$ vi Git.txt

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|MERGING)
$ git add Git.txt

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|MERGING)
$ git commit -m "Merge Conflict resolved"
[main e398b07] Merge Conflict resolved

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 771 bytes | 771.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Faisal1477/Git-practice.git
   aeb6302..e398b07  main -> main
branch 'main' set up to track 'origin/main'.
```

2. Recover Deleted Branch

Delete a local branch and then recover it using the reflog.

Step 1:- Switch to branch and push this branch to Repo

Step 2:- Then go to main branch and delete the other branch. Also push it to the Repo

Step 3:- Now use git reflog to fetch the logs. Then we can use the id when branch was deleted to recover it

Step 4:- Use git branch branch_name id. Then the branch will be recovered we can check the logs of that branch

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git switch Test
Switched to branch 'Test'

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git push origin Test
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Test' on GitHub by visiting:
remote:      https://github.com/Faisal1477/Git-practice/pull/new/Test
remote:
To https://github.com/Faisal1477/Git-practice.git
 * [new branch]      Test -> Test
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git branch -d Test
Deleted branch Test (was e398b07).

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push origin -d Test
To https://github.com/Faisal1477/Git-practice.git
 - [deleted]         Test
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git reflog
e398b07 (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: checkout: moving from Test to main
e398b07 (HEAD -> main, origin/main, origin/HEAD) HEAD@{1}: checkout: moving from main to Test
e398b07 (HEAD -> main, origin/main, origin/HEAD) HEAD@{2}: commit (merge): Merge Conflict resolved
36280d8 HEAD@{3}: checkout: moving from release to main
cad1348 (release) HEAD@{4}: commit: Changes in git in Release branch
aeb6302 HEAD@{5}: checkout: moving from main to release
36280d8 HEAD@{6}: commit: Changes in git in main branch
aeb6302 HEAD@{7}: clone: from https://github.com/Faisal1477/Git-practice.git
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git branch Test e398b07

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git switch Test
Switched to branch 'Test'

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git log --oneline --all --decorate --graph
*   e398b07 (HEAD -> Test, origin/main, origin/HEAD, main) Merge Conflict resolved
|\
| * cad1348 (release) Changes in git in Release branch
* | 36280d8 Changes in git in main branch
|/
| * d595873 (origin/release) Added new file conf
| * 5d6501d New file added in release branch
| * 65d7113 Revert "New file created"
| *   78b2d9b Merge branch 'main' of https://github.com/Faisal1477/Git-practice
| |\
| |/
|/|
* | aeb6302 New file created
| * 28aaf5c Revert "Removed hi file"
|/
* caddaf9 Removed hi file
* 6772b06 Initial setup
```

3.Undo Wrong Push

Push a wrong commit to GitHub, then undo it without losing history.

Step 1:- create a file and commit the changes.

Step 2:- Then push it to repo. After pushing the repo use revert and get the commit id from git log

Step 3:- After reverting the commit push the files again. Then process is success.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ echo "error file" > error

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ ls
Git.txt  Text  error  greet  hello

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git add error
warning: in the working copy of 'error', LF will be repla

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git commit -m "Error file is added"
[Test 8a8015d] Error file is added
 1 file changed, 1 insertion(+)
 create mode 100644 error
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git push -u origin Test
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 292 bytes | 292.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'Test' on GitHub by visiting:
remote:      https://github.com/Faisal1477/Git-practice/pull/new/Test
remote:
To https://github.com/Faisal1477/Git-practice.git
 * [new branch]      Test -> Test
branch 'Test' set up to track 'origin/Test'.
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git log -1
commit 8a8015dd294a9fa2d9a142574402e2fe10a87e51 (HEAD -> Test, origin/Test)
Author: Faisal1477 <mohdfaisal1477@gmail.com>
Date:   Wed Oct 28 14:36:15 2026 +0530

    Error file is added

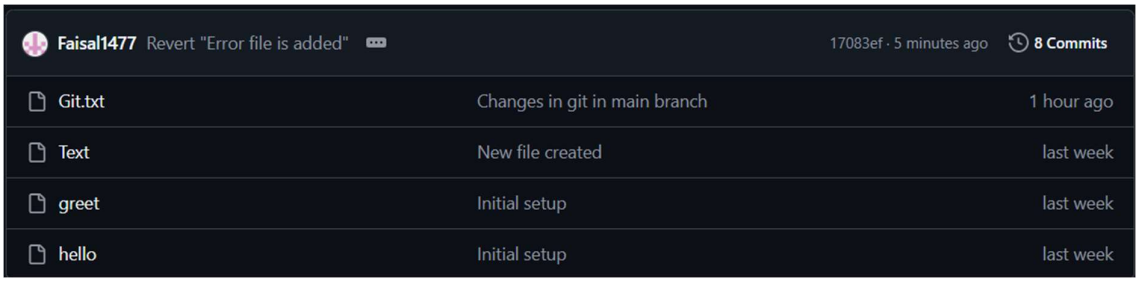
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git revert 8a801
[Test 17083ef] Revert "Error file is added"
 1 file changed, 1 deletion(-)
 delete mode 100644 error
```

```
UCRT64:/d/Git/Git-practice
Revert "Error file is added"

This reverts commit 8a8015dd294a9fa2d9a142574402e2fe10a87e51.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch Test
# Your branch is up to date with 'origin/Test'.
#
# Changes to be committed:
#       deleted:    error
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 266 bytes | 266.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Faisal1477/Git-practice.git
   8a8015d..17083ef  Test -> Test
```



4. Amend a Commit

Make a commit, then add a missing file to it using git commit --amend.

Step 1:- Create a file , add it to tracking. And make a commit

Step 2:- Now create a new file and add it to tracking.

Step 3:- Use git commit –amend. It open the previous commit and we can make changes in it. Save this and exit

Step 4:- Now use git log to check whether it is successfull or not. If we see single commit then it’s done

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ echo "This is new amend" > amend

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git add amend
warning: in the working copy of 'amend', LF will be replac

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git commit -m "New file amend added"
[Test 26c0795] New file amend added
 1 file changed, 1 insertion(+)
 create mode 100644 amend
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ echo "There are new amends" > amend_1

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git add amend_1
warning: in the working copy of 'amend_1', LF will be rep

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git commit --amend
[Test 57eb8d4] New file amend added
 Date: Wed Oct 28 15:04:53 2026 +0530
 2 files changed, 2 insertions(+)
 create mode 100644 amend
 create mode 100644 amend_1
```

```
UCRT64:/d/Git/Git-practice
New file amend added

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Wed Oct 28 15:04:53 2026 +0530
#
# On branch Test
# Your branch is ahead of 'origin/Test' by 1 commit.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#       new file:   amend
#       new file:   amend_1
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git log --oneline
57eb8d4 (HEAD -> Test) New file amend added
17083ef (origin/Test) Revert "Error file is added"
8a8015d Error file is added
e398b07 (origin/main, origin/HEAD, main) Merge Conflict resolved
cad1348 (release) Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup
```

5. Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.

Step 1:- Check the logs from different branches and select a commit id

Step 2:- Now switch to main branch and use cherry-pick and that id to apply it into main branch

Step 3:- after using ls we can see new files are added from other commit

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git log --oneline
57eb8d4 (HEAD -> Test) New file amend added
17083ef (origin/Test) Revert "Error file is added"
8a8015d Error file is added
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (Test)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ ls
Git.txt  Text  greet  hello

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git cherry-pick 57eb8d4
[main d5ced2d] New file amend added
 Date: Wed Oct 28 15:04:53 2026 +0530
 2 files changed, 2 insertions(+)
 create mode 100644 amend
 create mode 100644 amend_1

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ ls
Git.txt  Text  amend  amend_1  greet  hello
```

6.Interactive Rebase

Reorder and squash multiple commits into a single clean commit.

Step 1:- First view the commits we have with git log –oneline

Step 2:- Then use rebase command to merge the commits. It will open the text editor we have select base and squash other commits.

Step 3:- After saving and exiting this other text editor with commit message appears. Change it and save the file.

Step 4:- The rebase is done

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git log --oneline
d5ced2d (HEAD -> main) New file amend added
e398b07 (origin/main, origin/HEAD) Merge Conflict resolved
cad1348 (release) Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git rebase -i HEAD~3
Auto-merging Git.txt
CONFLICT (content): Merge conflict in Git.txt
error: could not apply cad1348... Changes in git in Release branch
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
You can amend the commit now, with

  git commit --amend

Once you are satisfied with your changes, run

  git rebase --continue
```

```
UCRT64:/d/Git/Git-practice
pick 36280d8 # Changes in git in main branch
squash cad1348 # Changes in git in Release branch
squash d5ced2d # New file amend added

# Rebase aeb6302..d5ced2d onto aeb6302 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#         create a merge commit using the original merge commit's
#         message (or the oneline, if no original merge commit was
#         specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
```

```
UCRT64:/d/Git/Git-practice
# This is a combination of 3 commits.
# This is the 1st commit message:

Changes in git in main branch and release branch. New file amend added

# This is the commit message #2:


# This is the commit message #3:


# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Wed Oct 28 15:27:57 2026 +0530
#
# interactive rebase in progress; onto aeb6302
# Last commands done (3 commands done):
#    squash cad1348 # Changes in git in Release branch
#    squash d5ced2d # New file amend added
# No commands remaining.
# You are currently rebasing branch 'main' on 'aeb6302'.
#
# Changes to be committed:
#       modified:   Git.txt
#       new file:   amend
#       new file:   amend_1
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|REBASE 2/3)
$ git add .

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|REBASE 2/3)
$ git commit -m "Conflict resolved"
[detached HEAD eedac76] Conflict resolved
 1 file changed, 4 insertions(+)

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|REBASE 2/3)
$ git rebase --continue
[detached HEAD 96c1236] Changes in git in main branch and release branch. New file amend added
 Date: Wed Oct 28 15:27:57 2026 +0530
 3 files changed, 6 insertions(+)
 create mode 100644 amend
 create mode 100644 amend_1
Successfully rebased and updated refs/heads/main.
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git log --oneline
96c1236 (HEAD -> main) Changes in git in main branch and release branch. New file amend added
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup
```

7.Tagging & Release

Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Step 1:- create a tag with git tag command. We can also see created with same command

Step 2:- Now push it to repo. We can see tag in github repo

Step 3:- Delete the tag with git tag -d tag_name. Also delete from the github with git push origin -d tag_name

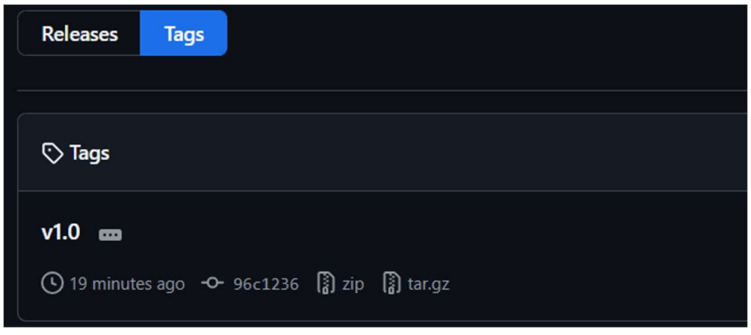
Step 4:- To restore this use git tag tag_name delete_id. It will be restored then push it to the github.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git tag

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git tag v1.0

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git tag
v1.0

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 522 bytes | 522.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Faisal1477/Git-practice.git
 * [new tag]         v1.0 -> v1.0
```




```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 96c1236)

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push origin -d v1.0
To https://github.com/Faisal1477/Git-practice.git
 - [deleted]         v1.0
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git tag v1.0 96c1236

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push origin -d v1.0
error: unable to delete 'v1.0': remote ref does not exist
error: failed to push some refs to 'https://github.com/Faisal1

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 522 bytes | 522.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Faisal1477/Git-practice.git
 * [new tag]         v1.0 -> v1.0
```

8.Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout.

Step 1:- Open an empty folder. And use clone with two flags first one is to ignore all the files initially which is –filter=blob:none.

Step 2:- Second flag is for enabling the sparse. The command should be like this git clone --filter=blob:none --sparse url

Step 3:- After this open the local repo. Then set the sparse-checkout to your sub-directory.

Step 4:- Now the clone of repo is complete. We can also check it with sparse-checkout list

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git
$ git clone --filter=blob:none --sparse https://github.com/Faisal1477/Git-practice.git
Cloning into 'Git-practice'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (20/20), done.
remote: Total 28 (delta 10), reused 25 (delta 8), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), done.
Resolving deltas: 100% (10/10), done.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), 164 bytes | 41.00 KiB/s, done.

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git
$ cd Git-practice

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|SPARSE)
$ git sparse-checkout set practice
remote: Enumerating objects: 2, done.
remote: Counting objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 2 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (2/2), 87 bytes | 87.00 KiB/s, done.

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|SPARSE)
$ ls
Git.txt  Text  greet  hello  practice/
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main|SPARSE)
$ git sparse-checkout list
practice
```

9.Reset vs Revert Challenge

Demonstrate the difference between git reset --hard and git revert in a repo.

1. git reset --hard

Moves the current branch to an earlier commit and discards changes from the working directory and staging area.

Suppose your repository has three commits:

- Commit A: Initial commit
- Commit B: Added a README file
- Commit C: Added a text file

If you perform a hard reset to Commit A:

- The current branch moves back to Commit A.
- Commits B and C are no longer part of the current branch's history.
- Tracked changes in the working directory and staging area are discarded.
- The changes from B and C are removed from the current version of the project.

The discarded commits may still be recoverable through Git's reflog for some time, but recovery is not guaranteed indefinitely.

2. git revert

Undoes the changes introduced by a particular commit by creating a new commit.

Suppose your repository has the same three commits:

- Commit A: Initial commit
- Commit B: Added a README file
- Commit C: Added a text file

If you revert Commit C:

- Git creates a new commit that reverses the changes introduced by Commit C.
- Commits A, B, and C remain in the project's history.

- The changes introduced by Commit C are undone in the current version.
- The repository's history remains intact.

If the changes conflict with later modifications, you may need to resolve conflicts before completing the revert.

10.Detached HEAD Challenge

Checkout a specific commit (detached HEAD state) and create a new branch from it.

Step 1:- Check git log and select a commit hash

Step 2:- Now use git checkout commit_hash. It will go into detached Head state.

Step 3:- Create a new branch while in this state.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git log --oneline
54de69c (HEAD -> main, origin/main, origin/HEAD) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ git checkout 54de69c
Note: switching to '54de69c'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 54de69c New file task added

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice ((54de69c...))
$ git switch -c feature
Switched to a new branch 'feature'
```

11. Git Hooks Challenge

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

Step 1:-Open a local repo. Then go to .git directory

Step 2:- Then go to Hooks directory. Create a file here

Step 3:- open the created file and write bash script

Step 4:- After completing and saving grant the permission to the file

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ ls -a
./  ../  .git/  Git.txt  Text  greet  hello  practice/  task.txt

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice (main)
$ cd .git/

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice/.git (GIT_DIR!)
$ ls
COMMIT_EDITMSG  HEAD  config  description  hooks/  index  info/  logs/  objects/  packed-refs  refs/

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice/.git (GIT_DIR!)
$ cd hooks/

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*      post-update.sample*      pre-merge-commit.sample*  pre-receive.sample*          sendemail-validate.sample*
commit-msg.sample*          pre-applypatch.sample*   pre-push.sample*          prepare-commit-msg.sample*   update.sample*
fsmonitor-watchman.sample*  pre-commit.sample*       pre-rebase.sample*        push-to-checkout.sample*

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice/.git/hooks (GIT_DIR!)
$ vi commit-msg
```

UCRT64:/d/Git/Git-practice/.git/hooks

```
#!/bin/bash

Msg_file="$1"
Msg=$(head -n 1 "$Msg_file")

if ! printf '%s\n' "$Msg" | grep -Eq '^JIRA-[0-9]+ '; then
        echo "Error: commit message must start with JIRA-XXX"
        echo "Ex:- JIRA-123 and login Validation"
        exit 1
fi
~
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/Git-practice/.git/hooks (GIT_DIR!)
$ chmod 755 commit-msg
```

12.Squash Merge vs Rebase Merge

Show the difference between squash merge and rebase merge with evidence.

I used ($ git merge –squash from Test branch for whatever committed in branch master it will copy in develop branch without committing in staging are then from there can do commit all file in single commit id and single commit message.

Git rebase will take all commits history in linear in details it will give clean tree for commie history from that branch.

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (Test)
$ git log --oneline
fcdf30b (HEAD -> Test) three files x, y, z added
54de69c (origin/main, origin/HEAD, main) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (Test)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git merge --squash Test
Updating 54de69c..fcdf30b
Fast-forward
Squash commit -- not updating HEAD
 x | 0
 y | 0
 z | 0
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 x
 create mode 100644 y
 create mode 100644 z
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git commit -m "Squash merge"
[main 66c67d9] Squash merge
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 x
 create mode 100644 y
 create mode 100644 z

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git log --oneline
66c67d9 (HEAD -> main) Squash merge
54de69c (origin/main, origin/HEAD) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git log --oneline
66c67d9 (HEAD -> main) Squash merge
54de69c (origin/main, origin/HEAD) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git rebase release
Successfully rebased and updated refs/heads/main.

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git log --oneline
26e778f (HEAD -> main) Squash merge
56eea32 (release) changes in A file
6e27d88 Three new files added
54de69c (origin/main, origin/HEAD) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup
```

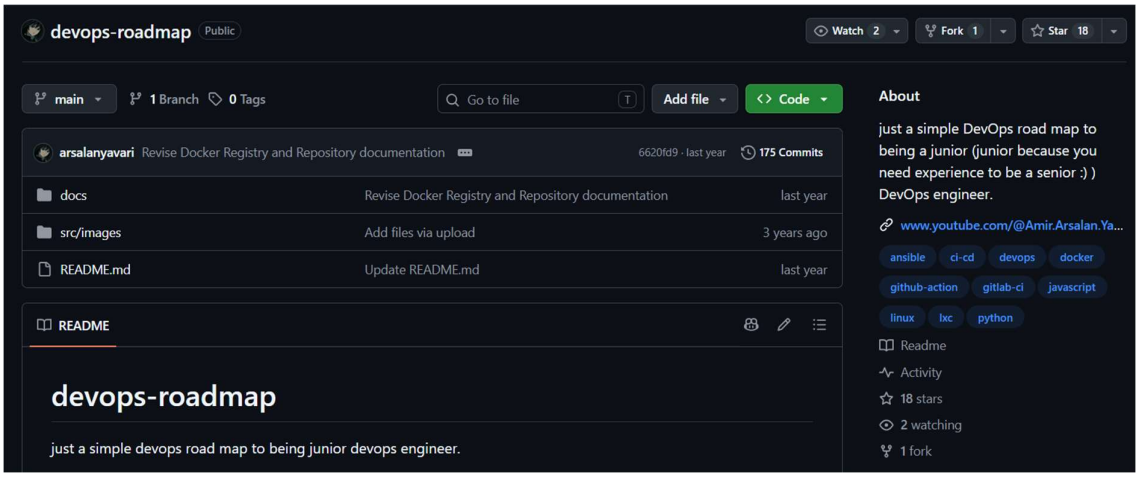
13. Fork & Pull Request Workflow

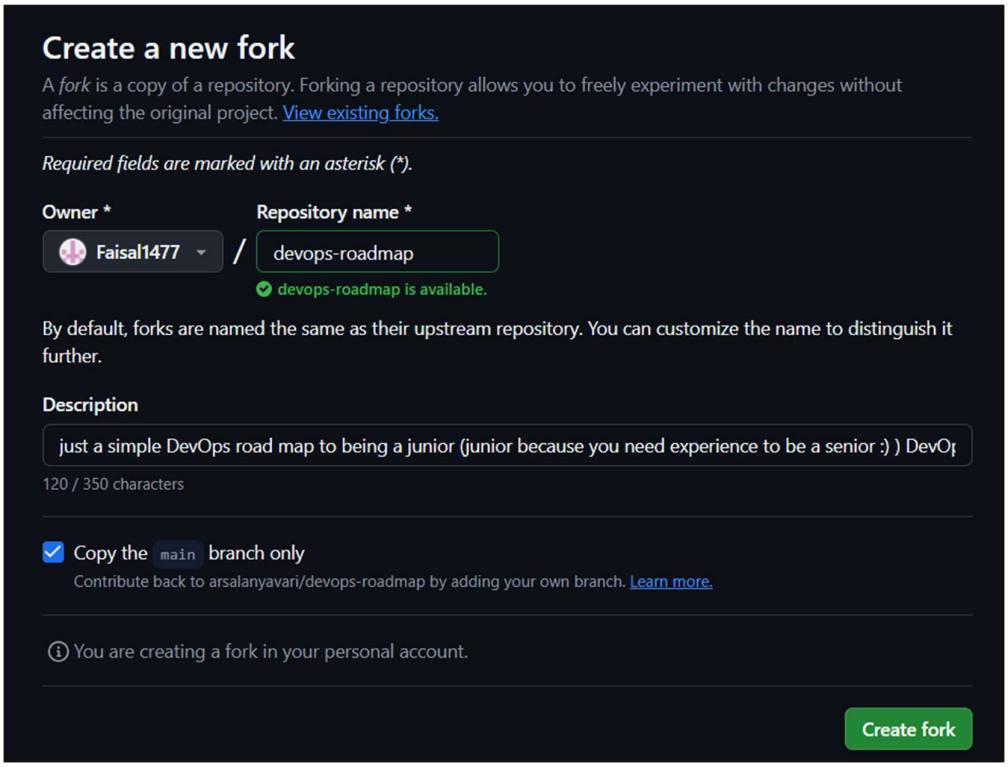
Fork a repo, make a change, and submit a pull request to the original repo.

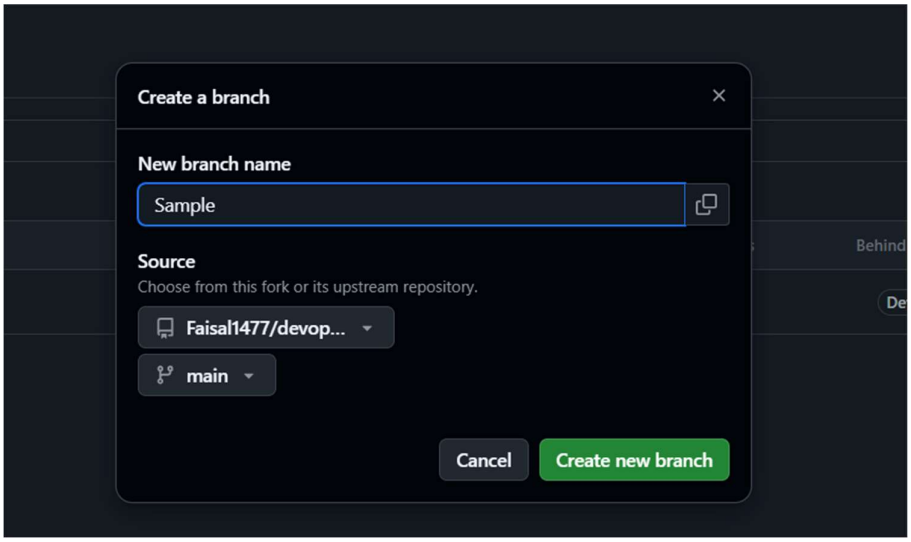
Step 1:- Take any repo and fork it so that It will appear in our repos

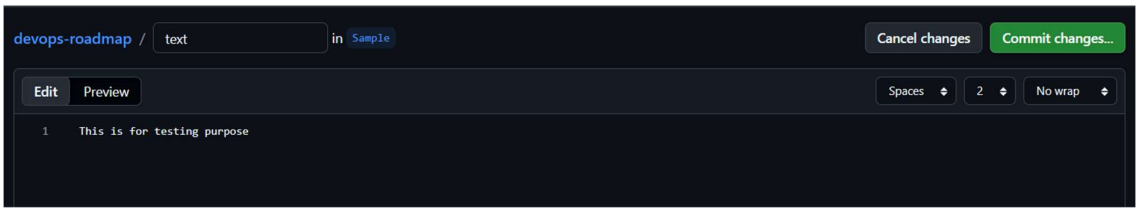
Step 2:- Then created new branch and add new file in that branch.

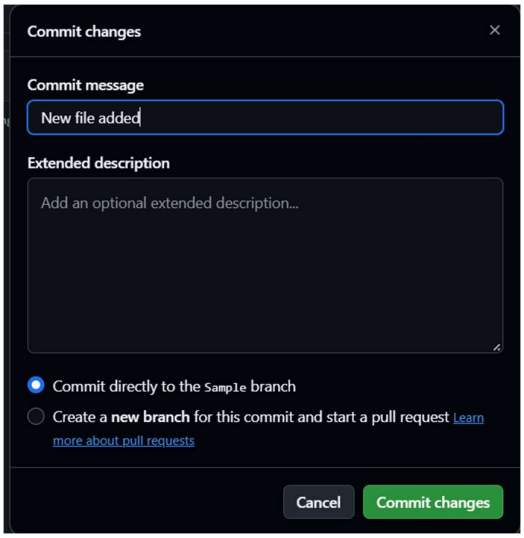
Step 3:- After that use Pull request

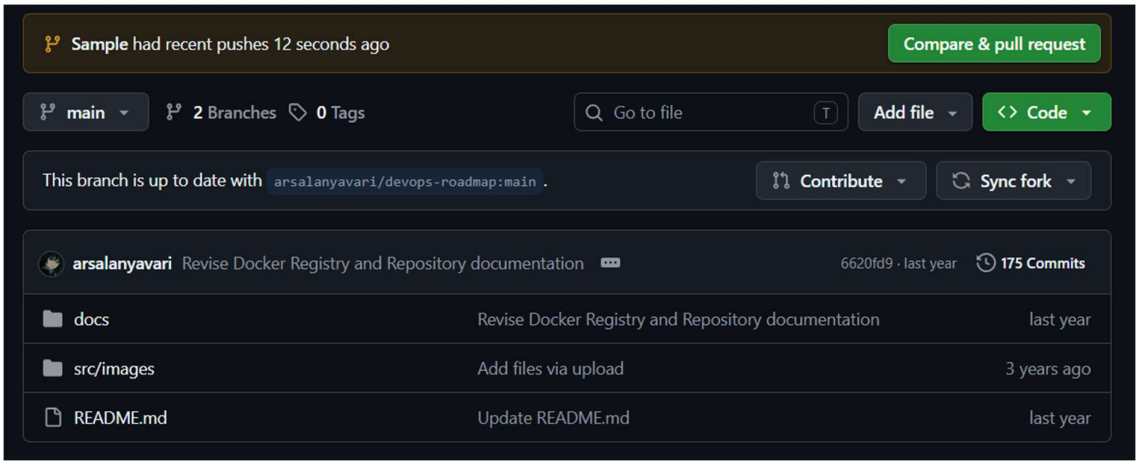


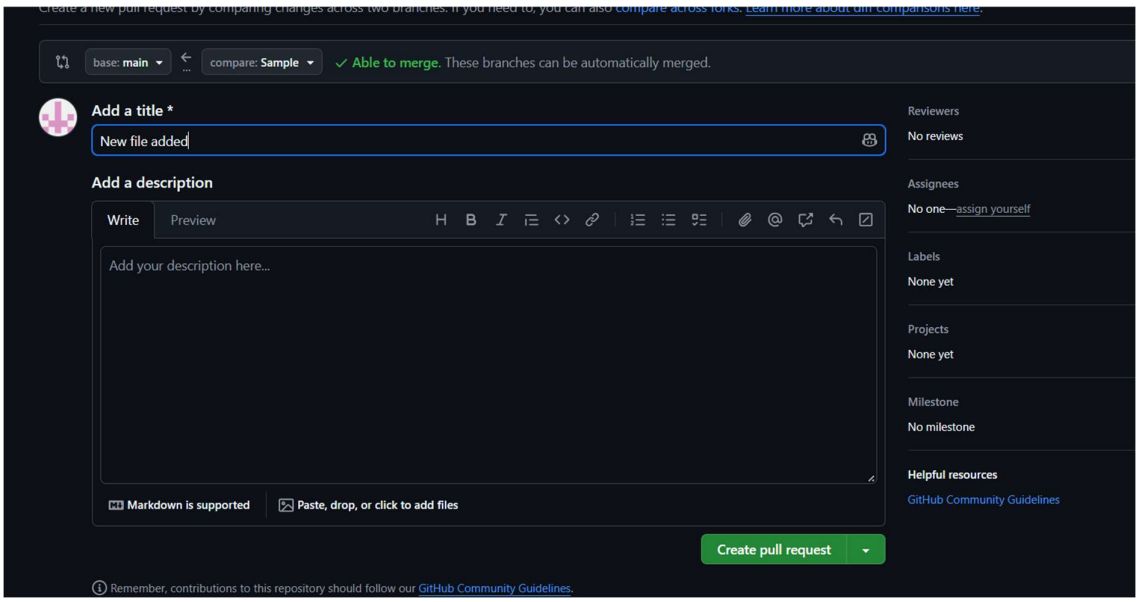


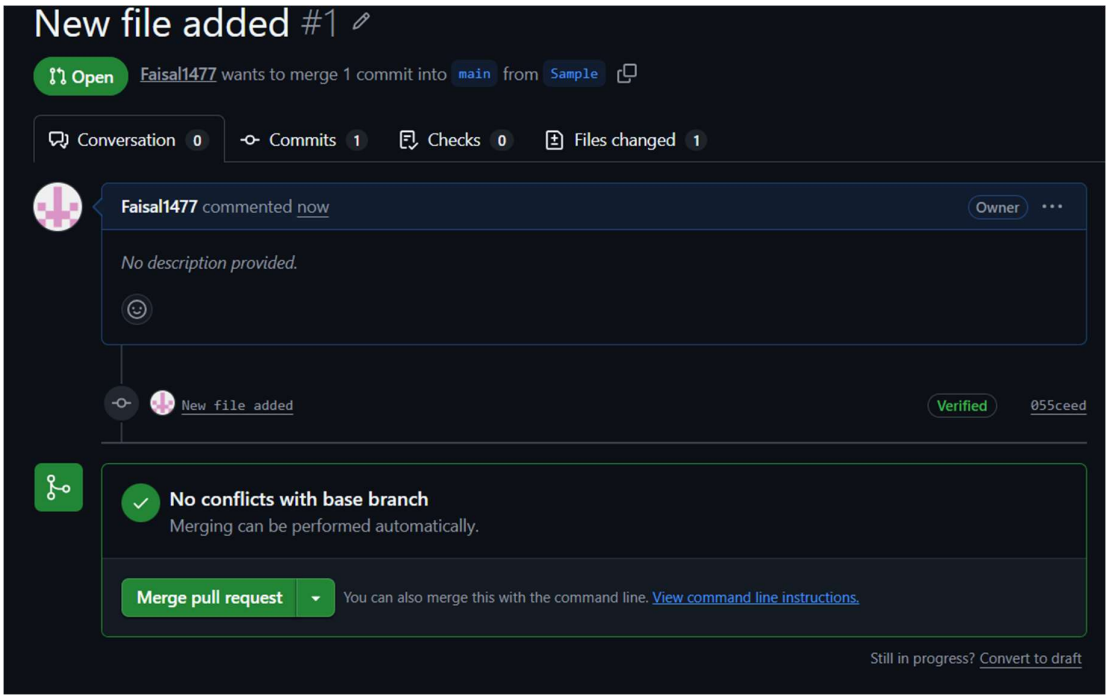


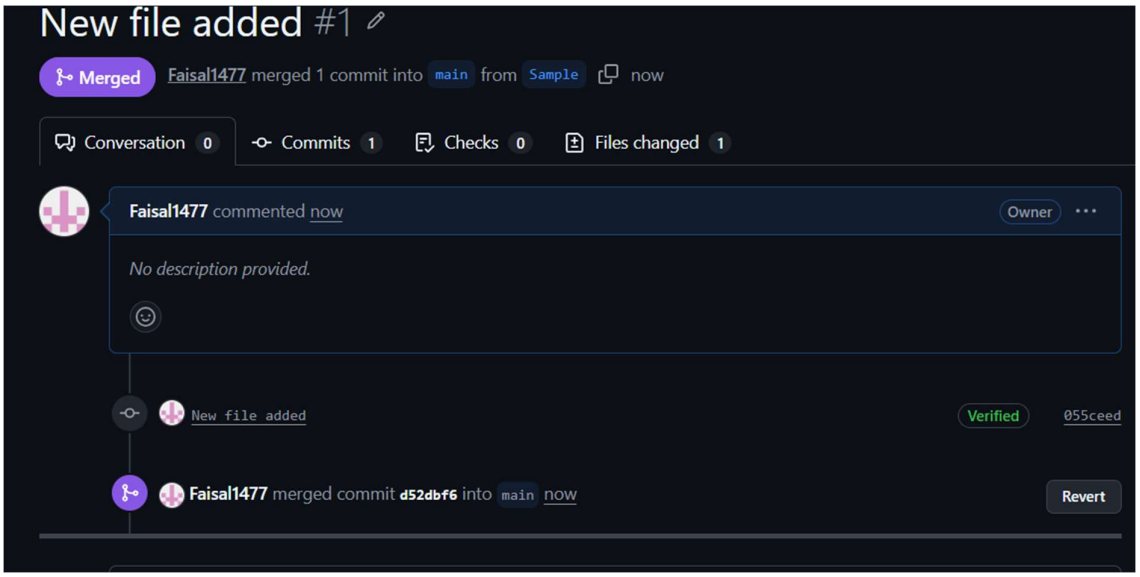

devops-roadmap / text in Sample
Cancel changes
Commit changes...
Edit
Preview
Spaces
2
No wrap
1 This is for testing purpose


Commit changes
Commit message
New file added
Extended description
Add an optional extended description...
Commit directly to the Sample branch
Create a new branch for this commit and start a pull request Learn more about pull requests
Cancel
Commit changes


Sample had recent pushes 12 seconds ago
Compare & pull request
main
2 Branches
0 Tags
Go to file
T
Add file
Code
This branch is up to date with arsalanyavari/devops-roadmap:main.
Contribute
Sync fork
arsalanyavari Revise Docker Registry and Repository documentation
6620fd9 · last year
175 Commits
docs
Revise Docker Registry and Repository documentation
last year
src/images
Add files via upload
3 years ago
README.md
Update README.md
last year


base: main
compare: Sample
Able to merge. These branches can be automatically merged.
Add a title *
New file added
Add a description
Write
Preview
Add your description here...
Markdown is supported
Paste, drop, or click to add files
Create pull request
Reviewers
No reviews
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Helpful resources
GitHub Community Guidelines
Remember, contributions to this repository should follow our GitHub Community Guidelines.





14.Recover Lost Commit

Commit something, reset hard, and then recover it using git reflog.

Step 1:- Create a commit first then use reset hard so that every thing of that commit is removed.

Step 2:- After that use git reflog to view every deleted or lost commits

Step 3:- now use reset hard with the commit id to bring everything of that commit back

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git reset --hard HEAD~1
HEAD is now at 26e778f Squash merge

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git reflog
26e778f (HEAD -> main) HEAD@{0}: reset: moving to HEAD~1
3e69048 HEAD@{1}: commit: Testing file added
26e778f (HEAD -> main) HEAD@{2}: rebase (finish): returning to refs/heads/main
26e778f (HEAD -> main) HEAD@{3}: rebase (pick): Squash merge
56eea32 (release) HEAD@{4}: rebase (start): checkout release
66c67d9 HEAD@{5}: commit: Squash merge
54de69c (origin/main, origin/HEAD) HEAD@{6}: checkout: moving from Test to main
fcdf30b (Test) HEAD@{7}: commit: three files x, y, z added
54de69c (origin/main, origin/HEAD) HEAD@{8}: checkout: moving from release to Test
56eea32 (release) HEAD@{9}: commit: changes in A file
6e27d88 HEAD@{10}: commit: Three new files added
54de69c (origin/main, origin/HEAD) HEAD@{11}: checkout: moving from Test to release
54de69c (origin/main, origin/HEAD) HEAD@{12}: checkout: moving from origin/release to Test
54de69c (origin/main, origin/HEAD) HEAD@{13}: checkout: moving from Test to origin/release
54de69c (origin/main, origin/HEAD) HEAD@{14}: checkout: moving from main to Test
54de69c (origin/main, origin/HEAD) HEAD@{15}: clone: from https://github.com/Faisal1477/Git-practice.git
```

```
Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git reset --hard 26e778f
HEAD is now at 26e778f Squash merge

Faisal@LAPTOP-4DO8SS4E UCRT64 /d/Git/New folder/Git-practice (main)
$ git log --oneline
26e778f (HEAD -> main) Squash merge
56eea32 (release) changes in A file
6e27d88 Three new files added
54de69c (origin/main, origin/HEAD) New file task added
722b889 Merge branch 'main' of https://github.com/Faisal1477/Git-practice
f2a0bef New folder added
96c1236 (tag: v1.0) Changes in git in main branch and release branch. New file amend added
e398b07 Merge Conflict resolved
cad1348 Changes in git in Release branch
36280d8 Changes in git in main branch
aeb6302 New file created
caddaf9 Removed hi file
6772b06 Initial setup
```